# Associative Context Memory: コンテキストウィンドウ依存を超える LLM コーディングエージェントのための永続的体験ベースメモリ

Hiroshi Yamato
Keio University / Signal compose Inc.
yamato@signalcompose.com

***Abstract*—大規模言語モデル（LLM）ベースのコーディングエージェントは根本的な制約を抱えている。そのメモリはコンテキストウィンドウに完全に束縛されており、トークン数の増加とともに品質が劣化する——Context Rot として知られる現象である。コンテキスト圧縮、セッション分割、検索拡張生成といった既存の緩和策は症状に対処するが、コンテキストウィンドウサイズへの根本的な依存を解決しない。本研究では Associative Context Memory (ACM) を提案する。これは LLM コーディングエージェントのための外部永続メモリアーキテクチャであり、暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉し、コンテキストウィンドウ容量に依存せずにセッション間で学習することを可能にする。本研究の主要な貢献は、コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系と強度序列である。インタラプトイベント（Ctrl+C / 強制停止）に続くインタラプト後の対話、rewind 操作、修正指示、中断なしのタスク完了を体系化する。ACM を既存の LLM コーディングエージェントエコシステムと互換性のある MCP（Model Context Protocol）サーバーとして設計し、コンテキストウィンドウ依存の低減と繰り返しセッションにおけるタスク性能の改善を評価するための実験設計を提案する。本論文は ACM をポジションペーパーおよびシステム提案として提示する。Phase 0 実現可能性プローブにより主要な実装前提を確認しつつ、完全な実験的検証は将来の研究に委ねる。**

## I. はじめに

Claude Code や OpenAI Codex に代表される LLM ベースのコーディングエージェントは、ソフトウェア開発タスクにおいて顕著な能力を示している。しかし、これらは根本的なアーキテクチャ上の制約を共有している。すなわち、すべてのコンテキスト——会話履歴、ツール出力、システムプロンプト、検索されたドキュメント——が有限のコンテキストウィンドウに収まる必要がある。セッション内でトークン数が増加するにつれ、モデルの性能は測定可能な形で劣化する。近年の実証研究はこの現象を **Context Rot** [1] として体系的に特徴づけ、トークン全体に注意が分散し、コンテキストが蓄積されるにつれてタスク成功確率が低下することを示している。

この問題に対する現在のアプローチは 2 つのカテゴリに分類される。

**圧縮・分割**アプローチ（auto-compact、Plan Mode、手動セッションリセット）は、Context Rot をセッション管理の問題として扱う。コンテキストサイズの削減により劣化を緩和するが、セッションをまたいで知識を引き継ぐ機構を持たない。新しいセッションはすべてゼロから始まる——我々はこれを「**健忘エージェント（amnesiac agent）問題**」と呼ぶ。

**検索拡張**アプローチ（Serena [2]、RAG ベースのコンテキスト拡張）は、エージェントの関連情報へのアクセスを改善するが、既存の成果物（コード、ドキュメント）の検索に焦点を当てており、何が成功し何が失敗したかという経験的知識の捕捉は行わない。

いずれのカテゴリも、より深い問いに答えていない。**エージェントは自身のインタラクション履歴から、コンテキストウィンドウとセッションをまたいで持続する形で学習できるか？**

人間の認知は示唆に富む対照を提供する。経験豊富な開発者は過去のデバッグセッションの逐語的な記憶を保持しない。代わりに連想パターンを蓄積する。「この種の認証エラーを見たとき、うまくいったアプローチは X、失敗したアプローチは Y。」この知識は疎で、圧縮されており、生のエピソード的詳細として保存されるのではなく、連想検索の手がかりによってインデックスされている。

本研究では、LLM コーディングエージェントのための永続的な外部メモリ層である **Associative Context Memory (ACM)** を提案する。ACM は以下を実現する。

1) 暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉する
2) 連想キーでインデックスされた構造化・検索可能な形式で体験を保存する
3) 過度なコンテキスト消費なしに、関連する過去の体験を新しいセッションに注入する
4) 持続的な性能のためのコンテキストウィンドウサイズへの依存を低減する

本研究の中心的な新規性は、コーディングエージェントインタラクションにおいて利用可能な**暗黙的フィードバックシグナルの分類体系**である。我々は、インタラプトイベント——Ctrl+C または同等の手段によるエージェント実行の強制停止——を、特にその直後の対話と組み合わせた場合に、ユーザー不満の最も強力なシグナルとして同定する。このシグナルは PTY（擬似端末）レベルで容易に観測可能であるにもかかわらず、エージェントフィードバックに関する先行研究では見過ごされてきた。

## II. 背景と関連研究

### A. コンテキストウィンドウの制約と *Context Rot*

Transformer アーキテクチャの self-attention 機構はコンテキストウィンドウ内のすべてのトークンに対して動作し、計算量はトークン数 $n$ に対して $O(n^2)$ でスケーリングする。Chroma Research [1] はこれを「Context Rot」として体系的に特徴づけ、Claude Opus 4 や Sonnet 4 を含む最先端モデルであってもコンテキストが充填されるにつれて測定可能な性能劣化を示すことを実証した。

実務者の間では、コンテキストウィンドウの最初の約 40%（「Smart zone」と呼ばれる）の範囲内で性能が最も信頼できるという経験則が観察されている [3]。

### B. コーディングエージェントにおけるコンテキスト管理

既存のコンテキスト管理戦略には以下が含まれる。

**Auto-compact**: コンテキストが容量に近づいた際のセッション履歴の要約。重大な弱点として、要約がコンテキスト容量の限界付近で動作する劣化したモデルによって実行される。

**Plan Mode (Claude Code)**: 計画と実装を、クリーンなコンテキストウィンドウを持つ別個のセッションに分離する。

**CLAUDE.md と永続的な指示** [4]: セッション開始時に読み込まれる静的ドキュメント。安定したプロジェクトコンテキストには有用だが、インタラクション履歴に適応しない。

### C. エージェントメモリシステム

LLM エージェントのための永続メモリの研究領域は急速に発展している。

**MemGPT / Letta** [5] は、コンテキストウィンドウを「RAM」、外部ストレージを「ディスク」として扱う階層型メモリアーキテクチャを導入した。

**Mem0 および A-MEM** [6] は、セマンティック検索とグラフベースの組織化を備えたメモリ層を提供する。A-MEM [7] は動的なノート構築とリンク生成を通じて柔軟性を向上させている。

**A-MAC** [8] は、ACM に構造的に最も類似した先行研究である。メモリ受入を 5 つの価値シグナルに分解するが、コーディングエージェントからの行動シグナルを利用しない。

**REMEMBERER** [9] は、Q 値を含むインタラクション記録としてエピソディックメモリを実装する。

**Self-Generated In-Context Examples** [10] は、成功したタスク軌跡を in-context examples として保存し、顕著な性能向上（73%→89%）を実証している。

**Letta Code** [11] は、MemGPT/Letta アーキテクチャに基づくメモリファーストのコーディングエージェントである。

**How Memory Management Impacts LLM Agents** [12] は、LLM エージェントの「experience-following property」を実証的に示している。

**Reflexion** [13] は、タスク試行の失敗後に言語的な自己フィードバックを生成し、エピソディックメモリとして保存する。ACM との重要な違いとして、Reflexion はエージェント自身の自己省察によってフィードバックを生成するのに対し、ACM はユーザーからの暗黙的行動シグナルを捕捉する。[1]

**ExpeL** [14] は、過去のタスク軌跡から転移可能な知識を抽出する。ACM はリアルタイムの暗黙的フィードバックシグナルに焦点を当てる点で異なる。

**Voyager** [15] は、Minecraft 環境における成功したタスク実行から永続的なスキルライブラリを構築する。

すべての先行メモリシステムに共通する制約は、**PTY レベルの行動シグナル**をファーストクラスのメモリ品質シグナルとして**欠いている**ことである。

### D. 人間-*LLM* インタラクションにおける暗黙的フィードバック

ICML 2025 の論文 [16] は、暗黙的ユーザーフィードバックを分析した。Google での並行研究 [17] は、Cider Chat のテレメトリを分析し、明示的フィードバックがインタラクションのわずか 0.6% でしか発生しないことを発見した。

いずれの研究も PTY レベルのインタラプトイベントをフィードバックシグナルとして検討していない。

### E. 検索拡張コード生成

**Serena** [2] は、LSP を使用してコードベースのセマンティックインデックスを構築する。ACM は Serena と直交する。

### F. *ACM* の位置づけ

表 Iに ACM と関連研究の比較を示す。

## III. ACM フレームワーク

### A. アーキテクチャ概要

ACM は MCP（Model Context Protocol）サーバーとして設計され、既存の LLM コーディングエージェントインフラストラクチャをラップする。

ACM 層は 3 つの機能を実行する：(1) セッション開始時の検索と注入、(2) セッション中のシグナル監視、(3) セッション終了時の体験書き込み。

### B. 体験エントリ構造

ACM は成功エントリ（「これをせよ」）と失敗エントリ（「これをするな」）の 2 種類を維持する。

### C. 検索と注入

各セッションの開始時に、ACM はユーザーの初期タスク記述からキーワードを抽出し、体験 DB のベクトル類似検索を行い、関連するエントリを圧縮して注入する。注入は意図的にコンパクトに保たれ、「コンテンツではなくインデックス」の原則を実装する。

### D. 体験スコアリングと昇格

セッション終了時に、ACM は利用可能なシグナルを用いてセッションをスコアリングし、昇格閾値以上のエントリを体験 DB に書き込む。

[1] Reflexion は単一タスク内のリトライ間でエピソディックメモリを永続化するが、異なるタスクやコードベース間での汎用的な検索可能ストアを提供しない。

TABLE I
ACM と関連エージェントメモリシステムの比較

| システム | クロスセッションメモリ | 失敗体験 | PTY/行動シグナル | コーディング特化 |
|---|---|---|---|---|
| MemGPT / Letta | Yes | No | No | No |
| Mem0 / A-MEM | Yes | No | No | No |
| A-MAC | Yes | Partial | No | No |
| REMEMBERER | Yes | Yes (Q 値) | No | No |
| Self-Gen ICE | Yes | No | No | No |
| Reflexion | No* | Yes | No | No |
| ExpeL | Yes | Yes | No | No |
| Voyager | Yes | No | No | No |
| Letta Code | Yes | No | No | Yes |
| **ACM（本研究）** | **Yes** | **Yes** | **Yes** | **Yes** |

* セッション内のみ。脚注参照。

## IV. 暗黙的フィードバックシグナル分類体系

本研究の中心的貢献は、LLM コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの体系的分類である。

### *A. Level 1:* インタラプト + インタラプト後対話（最高強度）

ユーザーがエージェント実行を強制停止し、その後に修正的な対話を行う。Claude Code の `PostToolUseFailure` フックが `is_interrupt` ブーリアンフィールドを提供する。

### *B. Level 2: Rewind* 操作（高強度）

ユーザーが rewind 機能を呼び出し、会話を以前の状態に復元する。

### *C. Level 3:* 修正指示（中強度）

ユーザーが会話内で自然言語による修正を提供する。

### *D. Level 4:* 中断なし完了（基準 / 弱い正のシグナル）

タスクがインタラプト、rewind、修正指示なしで完了する。

### *E.* シグナル強度のまとめ

以下の強度スコアは実験設計のための仮説的値である。序列はユーザーアクションのコストを反映する。

TABLE II
シグナル強度分類体系

| シグナル | スコア | 方向 | 捕捉方法 |
|---|---|---|---|
| インタラプト + 対話 | 0.90–1.00 | 負 | PTY + $N$ ターン |
| Rewind | 0.75–0.90 | 負 | セッションログ |
| 修正指示 (3+) | 0.60–0.80 | 負 | 会話テキスト |
| 修正指示 (1) | 0.30–0.50 | 負 | 会話テキスト |
| テスト通過 + 中断なし | 0.70–0.85 | 正 | 終了コード |
| 中断なし（テストなし） | 0.40–0.60 | 正 | セッションログ |

## V. 実験設計

### *A. Phase 0:* 実現可能性プローブ

完全な実験評価に先立ち、ACM の主要な技術的前提が Claude Code 環境内で実現可能であることを検証するための実現可能性プローブを実施した。

**Finding 1**: PTY インタラプトシグナルは公式 hooks API を介してアクセス可能である。

**Finding 2**: セッションログは体験エントリ生成に十分なデータを提供する。

**Finding 3**: セッション後のトークン使用量は回復可能である。

**Finding 4**: Rewind イベントには専用のフックがない。

### *B.* 研究課題

- RQ1 ACM はコンテキストウィンドウ依存を低減しつつ、タスク性能を維持・改善するか？
- RQ2 ACM-S、ACM-F、ACM-SF のうち、どのメモリ構成が最大の性能改善をもたらすか？
- RQ3 暗黙的フィードバックシグナルの種類は体験エントリ品質と相関するか？
- RQ4 ACM は人為的に制約されたコンテキストウィンドウサイズ下で持続的な性能を可能にするか？
- RQ5 ACM は Serena と補完的か？

### *C.* タスクスイート

SWE-bench [18] は単一セッションタスクであり、ACM のクロスセッションメモリが利点を提供する条件を含まないため、直接使用しない。

**タスク A**: マルチファイルバグ修正。**タスク B**: 仕様書からの機能追加。**タスク C**: 設計原則に基づくリファクタリング。

各タスクは条件あたり 5 回の繰り返しセッションで実行する。

## VI. 議論

### A. 健忘エージェント問題

現在の LLM コーディングエージェントは設計上、健忘的である。ACM は経験的知識をセッション間で永続化・蓄積する形式で外部化することにより、この問題に直接対処する。

### B. ファーストクラスシグナルとしてのインタラプト

インタラプトイベントはエージェントシステム研究においてフィードバック機構として十分に活用されてこなかった。コーディングエージェントにおいて、インタラプトは一般的であり意味的に豊かである。

### C. 強化学習との関係

ACM のシグナル分類体系は強化学習における報酬シグナルと構造的類似性を持つが、ACM は基盤モデルの重み更新を行わず、コンテキストレベルで動作する。

### D. 限界

**コールドスタート**: 新しいタスク種別での最初のセッションでは ACM は利点を提供しない。

**体験の陳腐化**: コードベースの進化が過去の体験を誤解を招くものにする可能性がある。

**インタラプトの曖昧性**: すべてのインタラプトがエージェントアプローチへの不満を示すわけではない。

**Rewind 検出**: Claude Code は rewind 操作用の専用フックイベントを提供していない。

**リアルタイムコンテキスト使用量**: アクティブセッション内のトークン数は hooks API を介してアクセスできない。

## VII. 結論

本研究では、暗黙的なユーザーフィードバックから成功・失敗体験を捕捉する LLM コーディングエージェントのための永続的外部メモリアーキテクチャである Associative Context Memory (ACM) を提示した。ACM の主要な貢献は以下の通りである。

1) コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系
2) 成功と失敗を区別する体験エントリ構造
3) MCP 互換のアーキテクチャ設計
4) クロスセッション学習とコンテキストウィンドウ依存低減を評価するための実験設計

論文草稿および関連ドキュメントはhttps://github.com/signalcompose/okitegami_paperで公開している。

### 著者貢献および *AI* 利用開示

本論文におけるすべてのリサーチクエスチョン、中核的アイデア、アーキテクチャ概念、および知的貢献は Hiroshi Yamato に帰属する。著者は本論文に記載された内容に対して全責任を負う。

本論文は人間主導の AI 協働ワークフローを通じて制作された。**Claude.ai** が Strategist（戦略担当）として、**Claude Code** が Implementer（実装担当）として機能した。AI が生成したテキストはすべて、掲載前に著者によるレビュー・承認を経ている。

## References


[1] Chroma Research, "Context rot: How increasing input tokens impacts LLM performance," https://research.trychroma.com/context-rot, 2025, technical report.

[2] Oraios AI, "Serena: A coding agent toolkit providing semantic code retrieval and editing via LSP and MCP," https://github.com/oraios/serena, 2025, open-source software.

[3] G. Huntley, "How to build a coding agent," https://ghuntley.com/agent/, 2025, blog post. Video: https://www.youtube.com/live/fOPvAPdqgPo.

[4] Anthropic, "Effective context engineering for AI agents," https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025, engineering blog post.

[5] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[6] P. Chhikara *et al.*, "Mem0: The memory layer for personalized AI," 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[7] W. Xu *et al.*, "A-MEM: Agentic memory for LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[8] G. Zhang, W. Jiang, X. Wang, A. Behr, K. Zhao, J. Friedman, X. Chu, and A. Anoun, "Adaptive memory admission control for LLM agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.04549

[9] Y. Zhang *et al.*, "REMEMBERER: Equipping large language models with persistent memory via long-term interaction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2306.07929

[10] V. Sarukkai *et al.*, "Self-generated in-context examples improve LLM agents for sequential decision-making," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: https://arxiv.org/abs/2505.00234

[11] Letta, "Letta code: Memory-first coding agent," https://www.letta.com/blog/letta-code, 2025, accessed 2026-03-08.

[12] Z. Xiong *et al.*, "How memory management impacts LLM agents: An empirical study of experience-following behavior," 2025. [Online]. Available: https://arxiv.org/abs/2505.16067

[13] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[14] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2308.10144

[15] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.16291

[16] Y. Liu, M. J. Q. Zhang, and E. Choi, "User feedback in human-LLM dialogues: A lens to understand users but noisy as a learning signal," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2507.23158

[17] D. Nam, M. Salawa, and S. Chandra, "Reading between the lines: Scalable user feedback via implicit sentiment in developer prompts," 2025. [Online]. Available: https://arxiv.org/abs/2509.18361

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770